

First Experiments on Automated Execution of Gherkin Test Specifications with Collaborating LLM Agents

Severin Bergsmann

Software Competence Center Hagenberg GmbH
Austria

Stefan Fischer

Software Competence Center Hagenberg GmbH
Austria

Alexander Schmidt

Software Competence Center Hagenberg GmbH
Austria

Rudolf Ramler

Software Competence Center Hagenberg GmbH
Austria

Abstract

Gherkin is a domain-specific language for describing test scenarios in natural language, which are the basis for automated acceptance testing. The emergence of Large Language Models (LLMs) has opened up new possibilities for processing such test specifications and for generating executable test code. This paper investigates the feasibility of employing LLMs to execute Gherkin test specifications utilizing the AutoGen multi-agent framework. Our findings show that our LLM agent system is able to automatically run the given test scenarios by autonomously exploring the system under test, generating executable test code on the fly, and evaluating execution results. We observed high success rates for executing simple as well as more complex test scenarios, but we also identified difficulties regarding failure scenarios and fault detection.

CCS Concepts

• **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → **Software testing and debugging**.

Keywords

LLMs, Domain Specific Language, Test Automation

ACM Reference Format:

Severin Bergsmann, Alexander Schmidt, Stefan Fischer, and Rudolf Ramler. 2024. First Experiments on Automated Execution of Gherkin Test Specifications with Collaborating LLM Agents. In *Proceedings of the 15th ACM International Workshop on Automating Test Case Design, Selection and Evaluation (A-TEST '24)*, September 19, 2024, Vienna, Austria. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3678719.3685692>

1 Introduction

Behavior-Driven Development (BDD) is an agile methodology that bridges technical and business stakeholders by capturing and testing requirements collaboratively [6]. In BDD, system behavior is specified through scenarios expressed in Domain Specific Languages (DSL) like *Gherkin*. Typically, these scenarios serve as requirement specifications and acceptance tests, ensuring the software behaves as expected.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

A-TEST '24, September 19, 2024, Vienna, Austria

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1109-1/24/09

<https://doi.org/10.1145/3678719.3685692>

Frameworks like *Cucumber* automate testing by mapping scenario steps to executable test code. However, this approach still requires developers or testers to implement and maintain the test code for each step, adding significant effort during development.

There is growing interest in using Natural Language Processing (NLP) and AI technologies, especially Large Language Models (LLMs), for automated testing [4]. LLMs, like OpenAI's GPTs, excel in processing and generating human-like text and executable code, offering new possibilities for test automation.

In this paper, we explore the use of LLMs to automate acceptance test scenarios for web-based applications described in natural language using the *Gherkin* DSL. Our goal is to run these scenarios without the need to implement any automation code for the test steps. Moreover, no additional information other than the URL of the web application is provided apart from the scenario descriptions. This means that the LLM needs to explore the system under test autonomously and also evaluate the execution results.

We achieve this by orchestrating multiple LLM agents with distinct roles. They collaborate in exploring the web application starting from the given URL, identifying suitable actions to accomplish the testing goal, and generating the related test code using a given test framework for web testing. We report the results from our first experiments running 15 *Gherkin* scenarios on a demo application. Our findings suggest that LLMs can indeed execute the given test scenarios and they show a promising performance, even for more complex scenarios which require multiple steps to execute. All results, code and chat logs are available in our online appendix¹.

2 Background

2.1 Gherkin and Cucumber

Gherkin is an actively used, human-readable DSL for BDD. It allows domain experts, testers, and developers to describe the behavior of software systems in form of structured scenarios [6]. The structure is based on the keywords *Given*, *When*, *Then* to define the steps of a test scenario. *Given* defines the initial context, *When* describes an action performed, and *Then* defines the expected outcome.

Cucumber is a testing framework that allows writing automated tests using the *Gherkin* DSL. The *Given-When-Then* steps are mapped to executable test code that performs the specified action on the system under test, if necessary, by including other test automation frameworks such as *Selenium*, *Playwright*, *Cypress*, etc.

¹https://github.com/software-competence-center-hagenberg/2024_A-Test_Gherkin_by_LLM

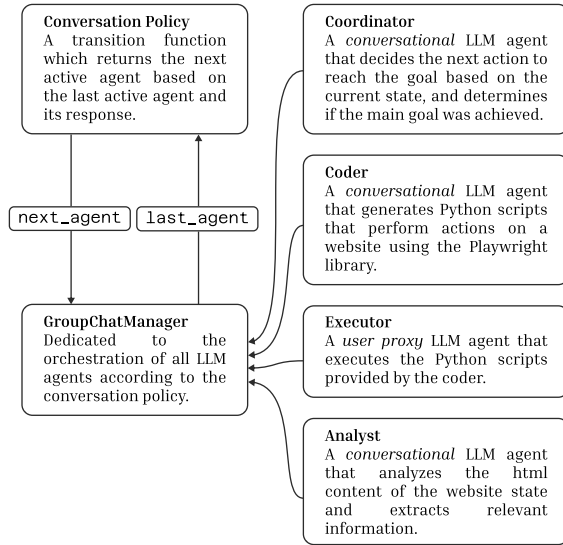


Figure 1: AutoGen Architecture: Each agent responds to the GroupChatManager agent which controls the GroupChat behavior using a pre-defined conversation policy.

2.2 AutoGen

The *AutoGen* framework is designed to facilitate the coordination and management of multiple LLM agents, each assigned a distinct role [5]. The framework efficiently distributes tasks among the LLM agents, to foster collaboration among them and assist in breaking down complex goals into manageable tasks. *AutoGen* provides a basic agent type called *Conversational Agent*, and several wrappers that extend the basic agent. The framework can be configured to terminate a task when a certain condition is met or a maximum number of interactions between agents has happened.

3 Test Execution with AutoGen

Given a test scenario with *Given-When-Then* steps as input, the goal is to leverage AI agents for automatically executing this scenario on the system under test without requiring human developers or testers implementing test code. Our approach is based on an *AutoGen* multi-agent configuration. In this setting, the agents are collaborating together in deriving tasks from test scenario descriptions, generating and executing test code on the fly, and analyzing the behavior of the system under test.

3.1 Agent Configuration

As shown in Figure 1, four LLM agents (*Coordinator*, *Coder*, *Executor*, *Analyst*) communicate with each other via a *GroupChatManager* dedicated to the orchestration of these agents. The interaction of the agents is determined by the *Conversation Policy*, which defines the order in which agents respond to messages in the chat.

We configure the LLM agents using two agent types provided by *AutoGen*: the *Conversational* and the *User-Proxy* agent type. The former implements the most basic *AutoGen* agent configuration

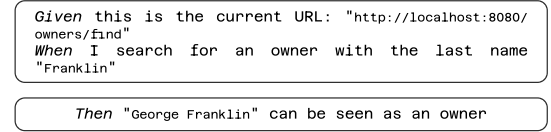


Figure 2: Given-When Pair (GWP) of Gherkin Scenario 1: The Given and When parts are served as agent input and the Then part is checked manually after the agents finished working.

which supports endpoint definition, human-in-the-loop, and function calling². The latter extends the *Conversational* agent with a given code execution configuration, allowing it to extract and execute code automatically. We developed system prompts for each agent to define its role as desired.

3.2 Input

The input given to the agents is a *Gherkin* scenario with test steps describing the tasks they have to perform. In our experiment, we decided to use the description of the *Then* steps of the *Gherkin* scenarios for evaluating whether the agents performed the tasks correctly. We therefore focus on automating the *When* steps of a *Gherkin* scenario in our experiment. The goals of test scenarios are passed as *Given-When Pairs (GWPs)* to the agents (Figure 2).

3.3 Execution Process

As first step, a browser session is spawned which is kept open in the background and can be accessed by the *Coder* agent throughout the entire process. Then the chat is initialized by the *Coordinator*, which sends the first message including the *GWP* the agents have to achieve (Figure 2) to the *GroupChatManager*. Using the *Conversation Policy* the manager now calls the respective agent, passes the whole chat history including the respective system prompt as context, and awaits its reply. The *Coder* is the second agent to be called and receives instructions from the *Coordinator*. It generates Python scripts to perform an action on the current browser session using the Python Playwright library³. The *Executor* extracts and executes the generated Python script which includes instructions to print the URL and HTML content after an action is taken. The console output of the *Executor* is processed by the *Analyst*. It extracts important information like interaction elements, contextual information about forms, and error messages from the HTML code that may help the *Coordinator* determine whether a next step is necessary or the main goal is reached. The obtained information allows the *Coordinator* to formulate more precise coding instructions for the *Coder*, resulting in better code [2]. This process is repeated until the *Coordinator* states that the main goal has been achieved or a maximum number of interactions between the agents is reached.

4 Experiment

We performed first experiments on using the multi-agent approach for automating test scenarios on a web-based demo application. Our goal is to demonstrate the feasibility of the proposed approach and to gather insights for refining and extending it for future use.

²https://microsoft.github.io/autogen/docs/reference/agentchat/conversable_agent/

³<https://github.com/microsoft/playwright-python>

4.1 Setup and Design

For our experiments we use the open-source sample application *PetClinic*⁴ as system under test, which is also frequently used in other testing studies [1]. We define 15 *Gherkin* scenarios to test different functionalities of the application via the Web interface. These include nine simple scenarios (covered by one *GWP*), three complex scenarios with multiple *When* steps, and three negative (faulty) tests that do not work on the application. The negative test scenarios simulate errors situations to assess the performance when scenarios fail. These involve unavailable actions, non-existent options (i.e., adding a pet lion), and inserting invalid data.

Each of the 15 scenarios is run ten times in isolation, with *PetClinic* reset between runs for consistency. Early experiments showed that LLMs often summarize multiple steps, failing to execute complex scenarios correctly. To address this, we split the three complex *Gherkin* scenarios into separate *GWPs*, with each *Given* step depending on the application's state after the previous steps. We retained browser sessions for executing the *GWPs* of a scenario.

Experiments use *gpt-3.5-turbo-0125* (GPT3.5) and *gpt-4o*⁵ (GPT4o) with identical system prompts. Each agent has common hyperparameters, including a *Temperature* of 0.2 to minimize output variability, crucial for the *Coder*, which outputs code based on a provided skeleton. We disable the *AutoGen* cache to prevent information flow between runs and set a request *timeout* of 600 seconds.

A run is considered successful if all follow-up *GWPs* deliver positive outcomes. Metrics from executing the *GWPs* are summed to measure overall performance. Failure of any *GWP* results in a failed run. We set limits of 50 turns and five code exceptions per *GWP*. Exceeding these limits results in a failed run.

4.2 Evaluation Metrics

We assess our approach's performance using the following metrics:

- **Passed Runs** The number of successful runs that the given scenario is correctly executed or aborted (for negative tests), depending on the test rationale.
- **Reported Success** The number of runs that the LLM agent system claimed the test scenario was executed successfully.
- **Execution Time** The time required by the approach to execute a test scenario.
- **Code Exceptions (#Exceptions)** The number of times the generated Python scripts results in an execution error.
- **Number Of Turns (#Turns)** The number of interactions between the agents to finish a test scenario. An interaction is defined as one agent-to-manager message.
- **Scripts Generated (#Scripts)** The number of Python scripts that were generated during the test scenario.
- **Consumed Tokens (#Tokens)** The number of tokens needed by all agents to complete a test scenario. We use the token count reported by the *AutoGen* framework.

5 Results & Discussion

Table 1 shows all metrics for each test scenario as average over the ten runs. The rows separated by lines relate to the nine simple

scenarios (S), the three complex ones (C), and the three negative test scenarios (N).

5.1 Reported Successful Runs

Column *Passed Runs* shows the number successful runs based on our manual evaluation of the execution results, *Reported Success* lists how often the agents reported a successful run. The difference between these numbers indicates hallucination by GPT3.5, especially the *Coordinator* agent. This agent incorrectly classified nine runs of simple scenarios and three runs of complex Scenario *C11* as successful. For example, in Scenario *S9*, the *Coordinator* incorrectly forwarded a different date for adding a new pet, marking the run as successful. Conversely, one run of Scenario *S7* was not reported as successful because the *Coordinator* missed a data field, even though it was displayed in the form. Additionally, six complex runs were not reported as successful, despite being correctly executed according to our manual evaluation.

5.2 Comparison of GPT3.5 and GPT4o

Overall, GPT3.5 achieved an 87.8% success rate on simple scenarios (*S1* to *S9*), with four scenarios working in all ten runs. For complex scenarios (*C10* to *C12*), the success rate dropped to 66.7%. The situation improved considerably with GPT4o, which successfully performed all simple and complex test scenarios and correctly reported success without hallucinations. On average, GPT4o performed more interactions, generated more scripts, and consumed more tokens. Detailed log analysis showed GPT4o agents executed finer-grained steps, generated better Python scripts, and accurately interpreted *GWPs* with a clear execution plan. Even on multi-page *GWPs* requiring navigation and context interpretation, GPT4o showed no confusion. However, GPT4o's higher token consumption resulted in a 4x cost increase compared to GPT3.5.

5.3 Negative Test Scenarios

The negative test scenarios reveal issues in both GPT models, since the tests assume false premises [3]. For Scenario *N13*, involving an impossible action in *PetClinic*, GPT3.5 tried different actions until hitting the exception limit in six runs, incorrectly reported success once, and correctly aborted three times. GPT4o correctly aborted all runs. For Scenario *N14*, GPT3.5 reported success in six runs by selecting the default option, ignoring the missing option, reaching the exception limit three times, and correctly aborting once. GPT4o correctly aborted six runs (but incorrectly selected the option "cat" instead of "lion") and reported success in four runs. The final scenario (*N15*) challenged both models, leading them to report success in all ten runs by altering the invalid input to be accepted, potentially missing a fault in input processing.

5.4 Lessons Learned

From our first experiments, we deduce following insights.

- **Precise specification of scenarios:** Large test scenarios involve complex interactions and dependencies, which were found to increase errors in automated execution dramatically. When breaking down scenarios into well-defined *GWPs*, we observed LLMs executing the steps independently and more accurately.

⁴<https://spring-petclinic.github.io/>

⁵<https://platform.openai.com/docs/models>

Table 1: LLM Gherkin scenario execution results: 3.5 refers to GPT3.5 and 4o refers to GPT4o (S: simple, C: complex, N: negative)

Scenario Nr	Passed Runs		Reported Success		Execution Time [s]		#Turns		#Scripts		#Exceptions		#Tokens	
	3.5	4o	3.5	4o	3.5	4o	3.5	4o	3.5	4o	3.5	4o	3.5	4o
S1	10	10	10	10	23.7	35.0	9.4	9.0	2.1	2.0	0	0	10033.6	10131.9
S2	7	10	9	10	51.3	120.6	16.6	25.8	3.9	6.2	0	0	29174.8	65173.6
S3	10	10	10	10	28.6	87.0	9.0	20.2	2.0	4.8	0	0	11475.2	38839.3
S4	7	10	10	10	36.8	55.0	13.8	12.2	3.2	2.8	0.2	0	19995.0	19498.0
S5	9	10	10	10	30.5	43.7	11.8	10.6	2.7	2.4	0	0	16996.4	15954.7
S6	7	10	10	10	29.9	47.7	11.4	10.2	2.6	2.3	0.1	0	15191.9	14860.1
S7	10	10	9	10	54.2	22.7	15.9	5.4	3.3	1.1	1.0	0	18260.1	5349.0
S8	10	10	10	10	26.1	26.0	10.2	5.4	2.3	1.1	0.2	0	10636.4	5491.6
S9	9	10	10	10	31.5	79.1	10.6	17.4	2.4	4.1	0	0	14830.7	34613.7
C10	7	10	7	10	289.1	519.2	81.6	113.4	18.9	26.6	1.4	0.1	117284.4	211811.9
C11	7	10	5	10	228.8	396.4	69.8	72.0	14.9	16.5	0.4	0	92032.4	115250.7
C12	6	10	5	10	220.4	172.4	54.7	36.0	12.8	8.0	2.9	0	70788.5	41384.3
N13	3	10	1	0	135.0	55.3	33.2	13.0	6.5	3.0	1.2	0	68339.1	19292.4
N14	1	6	6	4	136.1	80.2	18.3	17.4	4.4	4.1	2.5	0	26718.5	32133.7
N15	0	0	10	10	46.6	165.6	14.6	31.4	3.4	7.6	0	0	23666.4	89208.6

- **Effective prompt engineering:** Precisely defining the role and expected behavior of LLM agents through descriptive prompts is crucial. We were able to resolve many of the early execution problems simply by adjusting prompts.
- **Defining agent roles:** LLMs tend to mix up tasks and roles in multi-agent frameworks when clear role definitions are missing. Thus, non-ambiguous prompts are essential. Furthermore, we use separate agents even for seemingly trivial tasks because early experiments showed that assigning multiple tasks to agents increases confusion.
- **Beware of hallucination:** LLMs often ignore unlikely corner cases or negative scenarios, leading to hallucinations (e.g., inventing data or altering instructions). Both GPT models altered negative test scenarios to make them executable, resulting in false reports of successful execution, potentially missing real faults in testing.

6 Summary and Future Work

In this paper, we explored the potential of utilizing Large Language Models (LLMs) to execute *Gherkin* test scenarios for web application testing using Microsoft's *AutoGen* multi-agent framework. We defined a total of 15 test scenarios for the application *PetClinic* and employed an LLM agent system comprised of four agents with different roles. Our initial investigations demonstrate the feasibility of employing LLM agents to execute *Gherkin* scenarios.

Building on the insights gained from our first experiments, we identify several directions for future work. **Additional Experiments:** We will conduct more experiments across various systems and domains to evaluate our approach's generalizability and test different LLMs within a multi-agent framework. **Performance Improvement via Learning and Memory:** We plan to use an internal cache to store steps from web pages, reducing time and effort for repeated tasks and improving performance. **Automated Gherkin Scenario Decomposition:** We aim to automate the decomposition of longer test scenarios into smaller *GWPs*, a process

currently done manually. Automating this is crucial for executing any *Gherkin* scenario in an automated testing framework. **Efficient Context Preprocessing:** To prevent overwhelming the LLM with unnecessary data, we will develop an automated preprocessing step to filter relevant data, for efficient testing of complex applications.

By pursuing these avenues, we aim to overcome the limitations identified in our first experiments, enhance the robustness and generalizability of our findings, and contribute to the growing field of automated software testing using LLMs.

Acknowledgments

The research was supported by the Austrian Research Promotion Agency FFG (FFG grant no. 888127) and by the Austrian ministries BMK and BMAW and the State of Upper Austria in the frame of the SCCH COMET competence center INTEGRATE (FFG 892418).

References

- [1] Stefan Fischer, Pirmin Urbanke, Rudolf Ramler, Monika Steidl, and Michael Felderer. 2024. An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study. In *Conference on Automation of Software Test (AST)*. ACM. <https://doi.org/10.1145/3644032.3644445>
- [2] Tao Li, Chenhui Cui, Lei Ma, Dave Towey, Yujie Xie, and Rubing Huang. 2024. Leveraging Large Language Models for Automated Web-Form-Test Generation: An Empirical Study. *arXiv preprint arXiv:2405.09965* (2024).
- [3] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf
- [4] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *TSE* (2024). <https://doi.org/10.1109/TSE.2024.3368208>
- [5] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. *CoRR abs/2308.08155* (2023). <https://doi.org/10.48550/ARXIV.2308.08155> arXiv:2308.08155
- [6] Matt Wynne, Aslak Hellesoy, and Steve Tooke. 2017. *The cucumber book: behaviour-driven development for testers and developers*. Pragmatic Bookshelf.